

RTL with Verilog(Lab Week#3)

FIFO (First-In, First-Out)

FIFO stands for First-In-First-Out. It is a data storage structure where the first data written is the first data read out - similar to a queue or a line at a ticket counter.

In digital design, a FIFO is a hardware buffer used to temporarily store data between two systems operating at different speeds or with different clock domains.

Term Meaning

Write Storing data into the FIFO

Read Retrieving data from the FIFO

Depth Maximum number of data entries the FIFO can hold

Width Number of bits per data entry

Full Flag indicating FIFO has no space left

Empty Flag indicating FIFO has no data to read

Write Pointer Points to the next empty location to write

Read Pointer Points to the next location to read from

Types of FIFOs

1. Synchronous FIFO

Same clock for both write and read operations

Implementation:

Design:

```
module sync_fifo (
```

```
    input clk,
```

```
    input rst,
```

```
    input wr,
```

```
    input rd,
```

```
input [7:0] data_in,  
output reg [7:0] data_out,  
  
output reg full,  
output reg empty  
  
);
```

```
reg [7:0] mem [0:15];
```

```
reg [2:0] wr_ptr;  
reg [2:0] rd_ptr;
```

```
reg [4:0] cnt;
```

```
always @(posedge clk) begin
```

```
    if (rst) begin
```

```
        wr_ptr  <= 0;
```

```
        rd_ptr  <= 0;
```

```
        cnt     <= 0;
```

```
        data_out <= 0;
```

```
full    <= 0;  
empty   <= 1;
```

```
end
```

```
else begin
```

```
    // WRITE
```

```
    if (wr && !full) begin
```

```
        mem[wr_ptr] <= data_in;
```

```
        wr_ptr <= wr_ptr + 1;
```

```
    end
```

```
    // READ
```

```
    if (rd && !empty) begin
```

```
        data_out <= mem[rd_ptr];
```

```
        rd_ptr <= rd_ptr + 1;
```

```
    end
```

```
    // COUNT
```

```
    case ({wr && !full, rd && !empty})
```

```
2'b10: cnt <= cnt + 1;
```

```
2'b01: cnt <= cnt - 1;
```

```
default: cnt <= cnt;
```

```
endcase
```

```
// FULL
```

```
if (cnt == 15)
```

```
    full <= 1;
```

```
else
```

```
    full <= 0;
```

```
// EMPTY
```

```
if (cnt == 0)
```

```
    empty <= 1;
```

```
else
```

```
    empty <= 0;
```

```
end
```

```
end
```

```
endmodule
```

TestBench:

```
module sync_fifo_tb;

    reg clk;

    reg rst;

    reg wr;
    reg rd;

    reg [7:0] data_in;

    wire [7:0] data_out;

    wire full;
    wire empty;

    // DUT
    sync_fifo DUT (

        .clk      (clk),

        .rst      (rst),

        .wr       (wr),
        .rd       (rd),

        .data_in  (data_in),
```

```
.data_out (data_out),  
  
.full    (full),  
.empty   (empty)  
  
);
```

```
// CLOCK  
initial begin
```

```
    clk = 0;
```

```
    forever #5 clk = ~clk;
```

```
end
```

```
// TEST  
initial begin
```

```
    // RESET
```

```
    rst    = 1;
```

```
    wr     = 0;
```

```
    rd     = 0;
```

```
    data_in = 0;
```

```
#10;
```

```
rst = 0;
```

```
// WRITE A1
```

```
@(negedge clk);
```

```
wr    = 1;
```

```
data_in = 8'hA1;
```

```
// WRITE B2
```

```
@(negedge clk);
```

```
data_in = 8'hB2;
```

```
// WRITE C3
```

```
@(negedge clk);
```

```
data_in = 8'hC3;
```

```
// STOP WRITING
```

```
@(negedge clk);
```

```
wr = 0;
```

```
// READ A1
```

```
@(negedge clk);
```

```
rd = 1;
```

```
// READ B2
```

```
@(negedge clk);
```

```
// READ C3
```

```
@(negedge clk);
```

```
// STOP READING
```

```
@(negedge clk);
```

```
rd = 0;
```

```
#20;
```

```
$finish;
```

```
end
```

```
// MONITOR
```

```
initial begin
```

```

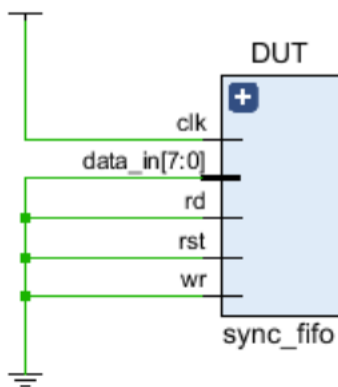
$monitor(
    "Time = %0t | wr = %b | rd = %b | data_in = %h | data_out = %h | full = %b | empty = %b",
    $time,
    wr,
    rd,
    data_in,
    data_out,
    full,
    empty
);

end

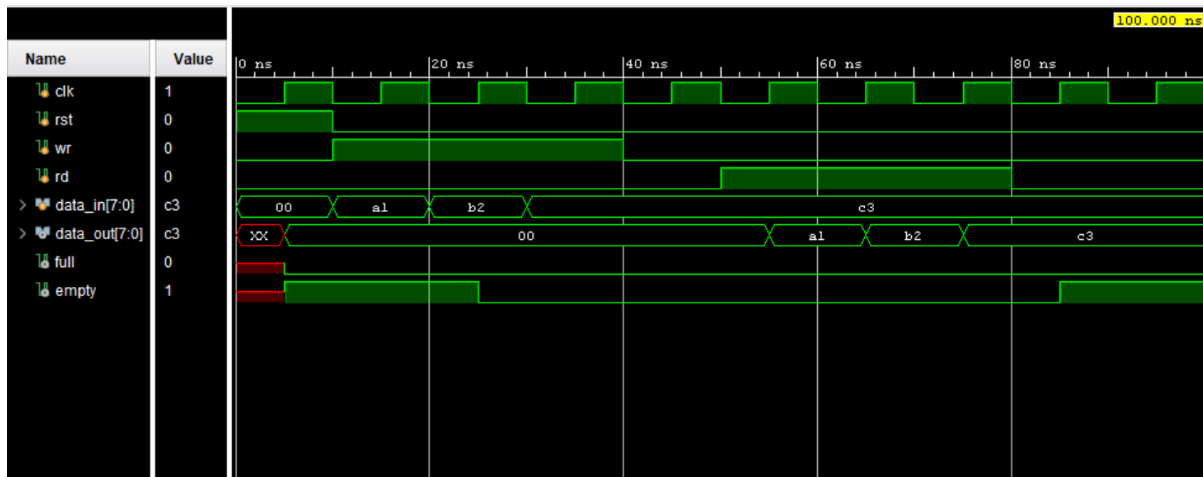
```

endmodule

Synthesis:



Wave Form:



2. Asynchronous FIFO

Different clocks for write and read operations

Requires Gray code and meta-stability handling

Used for crossing clock domains.

Implementation:

Design:

```
`timescale 1ns / 1ps
```

```
module async_fifo (
```

```
    input wr_clk,
```

```
    input rd_clk,
```

```
    input wr_rst,
```

```
    input rd_rst,
```

```
    input wr,
```

```
    input rd,
```

```
    input [7:0] data_in,
```

```
    output reg [7:0] data_out,
```

```

    output reg full,
    output reg empty
);
reg [7:0] mem [0:15];
reg [2:0] wr_ptr;
reg [2:0] rd_ptr;
reg [3:0] cnt;

// WRITE SIDE

always @(posedge wr_clk) begin

    if (wr_rst) begin

        wr_ptr <= 0;
        cnt  <= 0;
        full <= 0;

    end

    else if (wr && !full) begin

        mem[wr_ptr] <= data_in;

        wr_ptr <= wr_ptr + 1;

        cnt <= cnt + 1;
    end
end
end

```

```
// READ SIDE
```

```
always @(posedge rd_clk) begin
```

```
    if (rd_rst) begin
```

```
        rd_ptr  <= 0;
```

```
        data_out <= 0;
```

```
        empty   <= 1;
```

```
    end
```

```
    else if (rd && !empty) begin
```

```
        data_out <= mem[rd_ptr];
```

```
        rd_ptr <= rd_ptr + 1;
```

```
        cnt <= cnt - 1;
```

```
    end
```

```
end
```

```
endmodule
```

Test Bench :

```
module async_fifo_tb;
```

```
    reg wr_clk;
```

```
    reg rd_clk;
```

```

reg wr_rst;
reg rd_rst;

reg wr;
reg rd;
reg [7:0] data_in;
wire [7:0] data_out;
wire full;
wire empty;
// DUT
async_fifo DUT (

    .wr_clk  (wr_clk),
    .rd_clk  (rd_clk),

    .wr_rst  (wr_rst),
    .rd_rst  (rd_rst),

    .wr      (wr),
    .rd      (rd),

    .data_in (data_in),
    .data_out (data_out),

    .full    (full),
    .empty   (empty)

);
// WRITE CLOCK
initial begin

```

```

    wr_clk = 0;

    forever #5 wr_clk = ~wr_clk;

end

// READ CLOCK
initial begin

    rd_clk = 0;

    forever #7 rd_clk = ~rd_clk;

end

// TEST
initial begin
    // INITIAL VALUES
    wr_rst = 1;
    rd_rst = 1;
    wr     = 0;
    rd     = 0;

    data_in = 0;

    // RESET
    #20;

    wr_rst = 0;

```

```

rd_rst = 0;

// WRITE DATA A1
    @(negedge wr_clk);
    wr    = 1;
    data_in = 8'hA1;
// WRITE DATA B2
    @(negedge wr_clk);

    data_in = 8'hB2;

// WRITE DATA C3
    @(negedge wr_clk);
    data_in = 8'hC3;


// STOP WRITING
    @(negedge wr_clk);
    wr = 0;
// WAIT
    #30;
// READ DATA
    @(negedge rd_clk);
    rd = 1;
    @(negedge rd_clk);
    @(negedge rd_clk);
// STOP READING
    @(negedge rd_clk);

    rd = 0;

```

```

#20;

$finish;

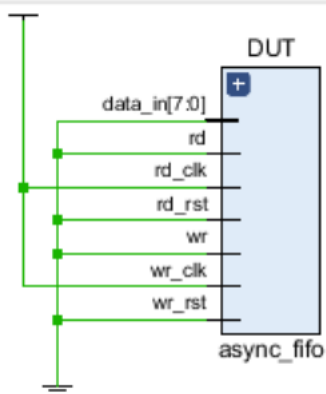
end

// MONITOR
initial begin

    $monitor(
        "Time = %0t | wr = %b | rd = %b | data_in = %h | data_out = %h | full = %b | empty =
        %b",
        $time,
        wr,
        rd,
        data_in,
        data_out,
        full,
        empty
    );
end
endmodule

```

Synthesis:



Wave Form:

